

Operating Systems: Core Concepts & Principles

Comprehensive University Lecture Revision Notes — CS301

Module Overview & Table of Contents

This document contains lecture material structured into 4 foundational units designed for revision and active recall:

Unit	Topic	Subtopics Covered	Page
1	Introduction to OS & Architecture	Dual-mode operations, Kernel vs User mode, System Calls	2
2	Process Management & Lifecycle	Process States, Process Control Block (PCB), Context Switching	3-4
3	CPU Scheduling Algorithms	FCFS, SJF, Priority, Round Robin, Multilevel Queue	5-6
4	Process Synchronization & Concurrency	Critical Section, Race Conditions, Semaphores, Mutex	7-8
5	Deadlocks & Memory Management	Deadlock Conditions, Banker's Algorithm, Paging, Virtual Memory	9-10

Unit 1: Introduction to Operating Systems

1.1 Definition and Role of an Operating System

An Operating System (OS) is system software that manages computer hardware, software resources, and provides common services for computer programs. It acts as an intermediary between the computer user and computer hardware.

Primary goals of an OS are convenience, efficiency, and resource utilization.

1.2 Dual-Mode Operation and Hardware Protection

To protect the operating system from errant users and to protect users from one another, modern computer hardware provides at least two separate modes of execution:

- **User Mode (Mode bit = 1):** The CPU executes user application code with restricted privileges. Direct hardware access and privileged machine instructions are prohibited.
- **Kernel Mode (Mode bit = 0):** Also known as supervisor mode or system mode. The CPU executes operating system core code with complete access to all hardware and execution of privileged instructions.

1.3 System Calls

System calls provide the programmatic interface between a running program and the operating system. When a user program requires a service from the kernel (such as reading a file or creating a socket), it performs a trap instruction causing a hardware mode switch from User Mode to Kernel Mode.

Examples of system calls in POSIX include: `fork()` (process creation), `exec()` (load binary), `read()/write()` (I/O operations), and `kill()` (signal delivery).

Unit 2: Process Management & Lifecycle

2.1 What is a Process?

A **process** is defined as a program in execution. While a program is a passive entity stored on disk (executable file), a process is an active entity with a program counter specifying the next instruction to execute and a set of associated system resources.

A process memory layout typically consists of four segments:

1. **Text Section:** The compiled executable machine code.
2. **Data Section:** Global and static variables.
3. **Heap Section:** Dynamically allocated memory during runtime (via malloc or new).
4. **Stack Section:** Temporary data such as function parameters, return addresses, and local variables.

2.2 Five-State Process Lifecycle Model

As a process executes, it transitions through distinct states:

- **New:** The process is being created and loaded into memory.
- **Ready:** The process is in main memory waiting to be assigned to a CPU by the scheduler.
- **Running:** Instructions are actively being executed on the CPU.
- **Waiting (Blocked):** The process is waiting for some event to occur (such as I/O completion or signal).
- **Terminated:** The process has finished execution and its resources are being deallocated.

Unit 2: Process Control Block & Context Switching

2.3 The Process Control Block (PCB)

Each process is represented in the operating system by a dedicated data structure called the **Process Control Block (PCB)** (also called a task control block). The PCB contains critical information associated with a specific process:

- **Process Identifier (PID):** Unique integer assigned to the process.
- **Process State:** Current state (Ready, Running, Waiting, etc.).
- **Program Counter (PC):** Address of the next instruction to be executed.
- **CPU Registers:** Accumulators, index registers, stack pointers, and condition codes.
- **CPU Scheduling Information:** Priority, scheduling queue pointers.
- **Memory-Management Information:** Base/limit registers, page tables or segment tables.
- **Accounting Information:** CPU time used, time limits.
- **I/O Status Information:** List of I/O devices allocated, open files.

2.4 Context Switching Mechanism

A **context switch** occurs when the CPU switches from executing one process to another. The operating system must save the state of the currently executing process to its PCB and reload the saved state of the new process from its PCB.

Performance Note: Context switch time is pure computational overhead because the system does no useful application work while switching. Context switch overhead typically ranges from 1 to 100 microseconds.

Unit 3: CPU Scheduling Algorithms

3.1 Scheduling Criteria

The objective of CPU scheduling is to maximize CPU utilization and system throughput while minimizing latency. Key performance metrics include:

- **CPU Utilization:** Percentage of time CPU is busy (ideal 40% to 90%).
- **Throughput:** Number of processes completed per unit time.
- **Turnaround Time:** Interval from submission to completion.
- **Waiting Time:** Total time spent sitting in the Ready Queue.
- **Response Time:** Time from submission until the first response is produced.

3.2 First-Come, First-Served (FCFS)

The process that requests the CPU first is allocated the CPU first using a FIFO queue. Non-preemptive.

***The Convoy Effect:** A major drawback of FCFS where short processes get stuck waiting behind one long, CPU-heavy process, drastically increasing average waiting time.*

3.3 Shortest-Job-First (SJF) & SRTF

Associates each process with the length of its next CPU burst. SJF is mathematically optimal because it gives the minimum average waiting time for a given set of processes. The preemptive version is called Shortest-Remaining-Time-First (SRTF).

Unit 3: Preemptive & Complex Scheduling

3.4 Round Robin (RR) Scheduling

Designed specifically for time-sharing systems. A small unit of time, called a **time quantum (or time slice)** (typically 10 to 100 milliseconds), is defined. The ready queue is treated as a circular FIFO queue.

- If process burst < quantum: Process releases CPU voluntarily.
- If process burst > quantum: Timer interrupts process, context switch occurs, and process is appended to the tail of the ready queue.

Rule of Thumb: About 80% of CPU bursts should be shorter than the time quantum to prevent excessive context switching overhead.

3.5 Priority Scheduling & Starvation

A priority number is assigned to each process; CPU is allocated to the process with the highest priority. A major problem is **indefinite blocking (starvation)**, where low-priority processes never execute.

Solution — Aging: A technique of gradually increasing the priority of processes that wait in the system for a long time.

3.6 Multilevel Feedback Queue (MLFQ)

Separates ready queue into multiple priority queues. Allows a process to move between queues based on CPU burst characteristics, favoring I/O-bound interactive processes.

Unit 4: Process Synchronization

4.1 The Critical-Section Problem

When multiple concurrent processes access and manipulate shared data, the outcome depends on the particular order of execution—this is a **race condition**. The section of code where shared resources are accessed is called the **Critical Section**.

A valid solution to the critical-section problem must satisfy three strict requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other processes can be executing in their critical sections.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not in their remainder section can participate in deciding who enters next.
3. **Bounded Waiting:** There must be a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter.

4.2 Mutex Locks

A mutex (Mutual Exclusion) lock is a synchronization tool with a boolean variable indicating if lock is available. Provides `acquire()` and `release()` operations.

Spinlocks: Mutex locks where a process loops continuously while waiting (busy waiting). Useful in multiprocessor systems when locks are held for short durations.

Unit 4: Semaphores & Synchronization Problems

4.3 Semaphores (Counting and Binary)

A **Semaphore** S is an integer variable accessed only through two atomic operations: `wait()` (or `P()`) and `signal()` (or `V()`).

- **Binary Semaphore:** Value ranges only between 0 and 1 (functions like a mutex lock).
- **Counting Semaphore:** Value can range over an unrestricted domain, used to control access to a finite pool of identical resources.

4.4 Classical Problems of Synchronization

- **Bounded-Buffer Problem (Producer-Consumer):** Producers write items into a shared buffer of size N while consumers remove items. Uses mutex, empty, and full semaphores.
- **Readers-Writers Problem:** Multiple readers can read simultaneously, but a writer must have exclusive access.
- **Dining Philosophers Problem:** Illustrates allocation of multiple shared resources without deadlock and starvation.

Unit 5: Deadlocks & Prevention

5.1 Deadlock Characterization

A **deadlock** is a situation where every process in a set is waiting for an event that can be caused only by another process in the set.

A deadlock can arise if and only if all four **Coffman Conditions** hold simultaneously:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources held by others.
3. **No Preemption:** Resources cannot be preempted; they can be released only voluntarily by the holding process.
4. **Circular Wait:** A closed chain of processes exists such that P₀ waits for P₁, P₁ waits for P₂, and P_n waits for P₀.

5.2 Deadlock Avoidance — Banker's Algorithm

The Banker's algorithm tests for safety by simulating the allocation of predetermined maximum possible amounts of all resources, before deciding whether allocation should be allowed.

Unit 5: Memory Management & Virtual Memory

5.3 Paging and Address Translation

Paging is a memory-management scheme that permits physical address space of a process to be non-contiguous. It eliminates external fragmentation.

- Physical memory is divided into fixed-sized blocks called **Frames**.
- Logical memory is divided into blocks of the same size called **Pages**.

The CPU generates a logical address divided into Page Number (p) and Page Offset (d). The **Page Table** maps page numbers to frame numbers in physical RAM.

5.4 Virtual Memory & Demand Paging

Virtual memory separates user logical memory from physical memory, allowing execution of processes not completely in RAM. In **Demand Paging**, pages are loaded into memory only when accessed.

- **Page Fault:** *An interrupt raised when a program accesses a page with an invalid bit (not in physical memory). The OS brings the missing page from disk into a free frame.*
- **Thrashing:** *High paging activity where a process spends more time swapping pages in and out than executing instructions. Occurs when total working set sizes exceed physical RAM.*